

Bangladesh University of Engineering and Technology (BUET)

Department of Computer Science and Engineering

Design Report

Tool 9: Dictionary Attack and Known Password Attack

Course: CSE406: Computer Security Sessional

Assignment: Attack Tool 9

Report deadline: 11th Week

Group Members

Member	Student ID	Responsibility
Arnob Biswas	2105015	Webapp + defense mechanism
Zaki Rehnoom Unmona	2105016	Attacker tools + report diagrams

September 11, 2026

Contents

1	Overview of the Project Idea	1
2	Network Topology, Components, and Timing Diagrams	1
2.1	Topology	1
2.2	Components	2
2.3	Timing Diagrams	3
2.3.1	Normal (legitimate) signup and login: browser flow	3
2.3.2	Dictionary attack: repeated exchange, one account	4
2.3.3	Known-password (spray) attack: repeated exchange, many accounts	4
2.3.4	With the lockout defense active	5
3	Packet and Frame Structure	5
3.1	Request (attacker or browser $\rightarrow$ server), one per login attempt	5
3.2	Responses, by outcome	6
3.3	Design Notes, and What the Packet Capture Is Expected to Show	6
4	Implementation Plan	6
5	Design Justification	7
6	Expected Outcome of a Successful Attack	8
7	Defense Idea	9
8	Conclusion	9

1 Overview of the Project Idea

In this project we implement and demonstrate two related password-guessing attacks, along with a working defense against them:

- **Dictionary attack:** the attacker targets *one specific account* and tries a long, ordered list of candidate passwords (a “dictionary” of common, likely passwords) against it, one at a time, until the server accepts one or the list runs out. This works because many users choose passwords that show up in common wordlists.
- **Known password attack** (password spraying, or credential reuse): instead of guessing many passwords against one account, the attacker starts from a small set of *known* passwords, such as defaults, passwords leaked in earlier breaches, or passwords picked up from another attack like sniffing, and tries each one against a *wide list of accounts*. This exploits password reuse and default credentials rather than weak individual passwords, and it is harder to catch with a simple per-account lockout, since no single account ever receives many failed attempts.

Following the assignment brief and our supervisor’s guidance, we are attacking a vulnerable application we built ourselves rather than an existing service such as SSH or FTP: a small signup/login web application built with Flask and SQLite. The *victim action* for the demo is a normal user signing up and logging in through a browser. The *attacker action* is the two attacks above, driven by HTTP client code we wrote ourselves, not an existing brute-forcing tool such as Hydra or Medusa. Alongside the attacks, we are building a defense mechanism, account lockout combined with slow password hashing, so the report can show the attacks being stopped, not just carried out.

We already have a working single-host prototype of the webapp, both attacker scripts, the seed data, and the lockout/slow-hash defense in this repository (`src/`), and we have run it manually against `localhost` in both the undefended and defended configurations. What is still missing is the two-VM network setup described in Section 2 and the measured timing/packet-capture evidence described in Section 6, both of which belong in the Final Report. This Design Report commits, in advance, to what we expect that evaluation to show.

2 Network Topology, Components, and Timing Diagrams

2.1 Topology

Development and the final demo will use the following topology: two VMs on a host-only virtual network, plus a legitimate browser client exercising the same webapp.

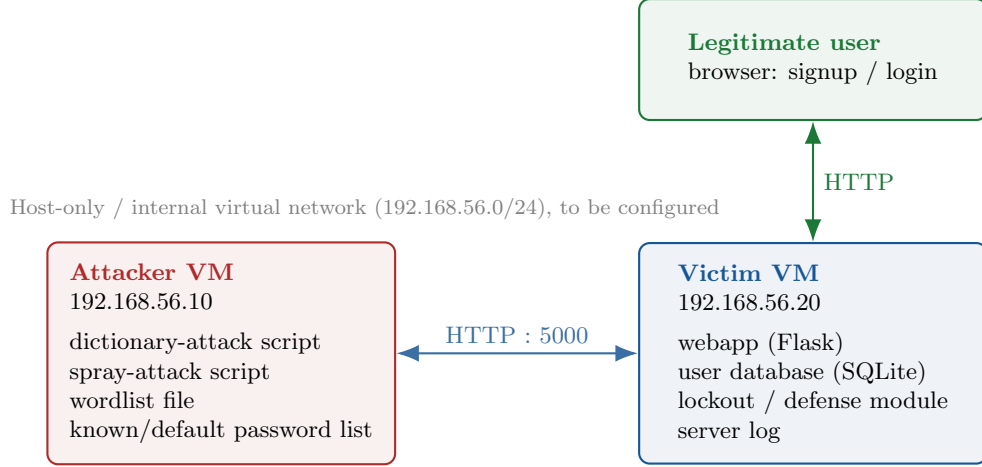


Figure 1: Attack topology: two VMs on a host-only virtual network, plus a legitimate browser client exercising the same webapp.

The IP addresses shown above are placeholders for the VirtualBox host-only subnet. We will replace them with the real addresses, along with an `ip addr` screenshot from both machines, once the VMs are set up.

2.2 Components

Component	Role
Victim webapp (Flask)	Serves signup, login, and dashboard pages; the attack target
User database (SQLite)	Stores usernames, salts, and password hashes
Defense module	Tracks failed login attempts per account and enforces temporary lockout; also selects the slow-hash mode
Dictionary-attack script	Attacker tool: many candidate passwords against one account
Spray-attack script	Attacker tool: a few known passwords against many accounts
Wordlist / known-password list	Input data for the two attacker scripts
Server / attacker logs	Record every login attempt and outcome, for the Final Report

Table 1: System components and their roles.

2.3 Timing Diagrams

The diagrams below describe the protocol we are implementing and the attack strategies we plan to run against it. This is the intended design, and we will verify it once the two-VM setup is in place. Request arrows (client → server) are shown in steel blue; responses are colored by outcome: green for success, amber for a failed attempt, and red for a logout.

2.3.1 Normal (legitimate) signup and login: browser flow

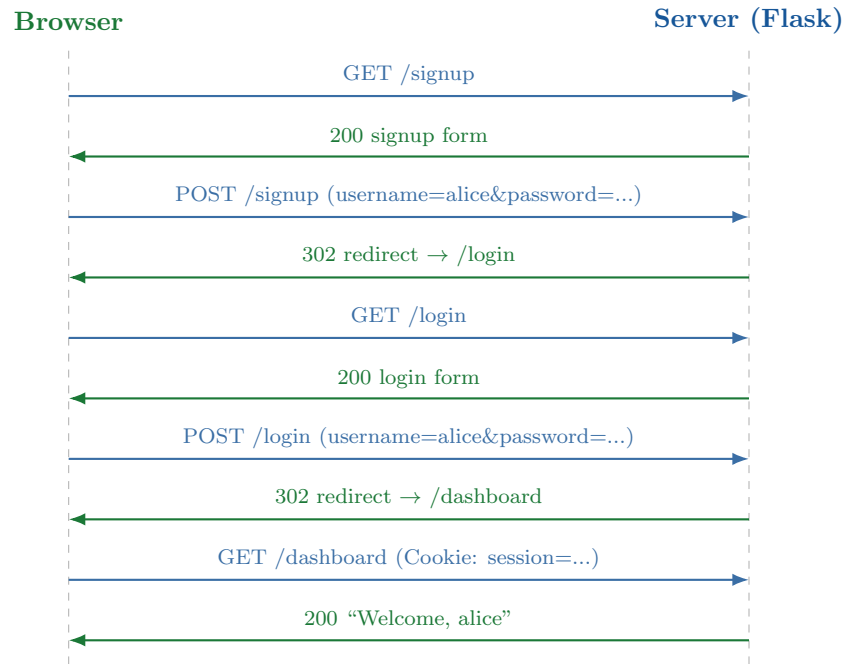


Figure 2: Normal protocol timing: one legitimate signup followed by one login.

2.3.2 Dictionary attack: repeated exchange, one account

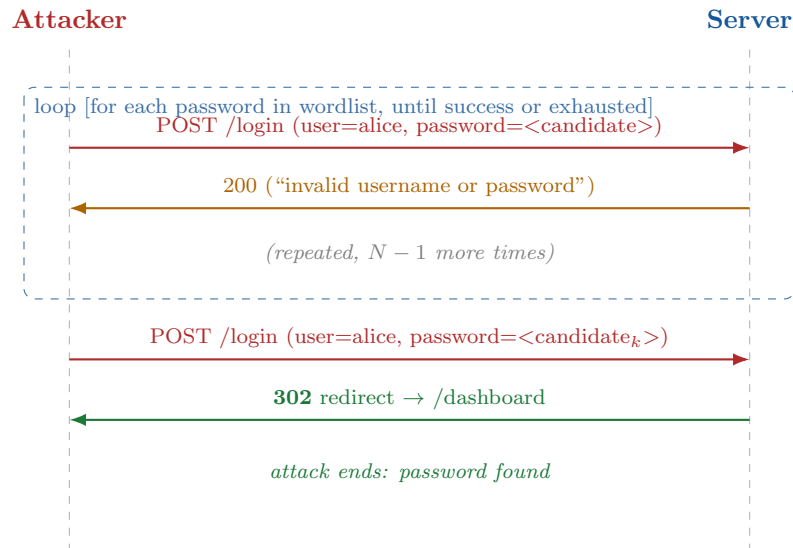


Figure 3: Dictionary attack: many candidate passwords tried in sequence against a single account.

2.3.3 Known-password (spray) attack: repeated exchange, many accounts

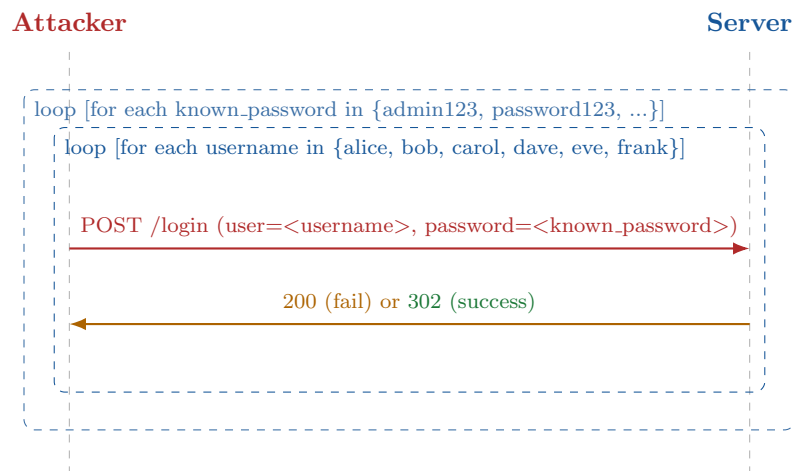


Figure 4: Known-password / spray attack: few known passwords tried across many accounts.

2.3.4 With the lockout defense active

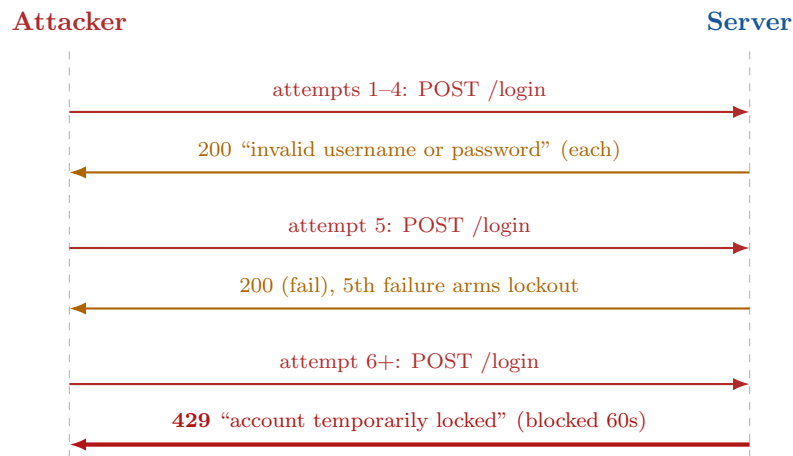


Figure 5: Lockout defense: the dictionary/spray attack should be blocked after 5 failed attempts within a 30-second window.

3 Packet and Frame Structure

The transport is standard TCP carrying HTTP/1.1, using Flask’s built-in development server. We do not need to craft raw Ethernet or IP frames for this attack class, since the exploit sits at the application layer. What we are designing from scratch is the *application behaviour and payload* the HTTP request carries, and how the server is meant to interpret it, not an existing authentication protocol implementation. The exact request/response format below is our intended design for the login endpoint.

3.1 Request (attacker or browser → server), one per login attempt

```
POST /login HTTP/1.1
Host: 192.168.56.20:5000
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

username=alice&password=sunshine
```

3.2 Responses, by outcome

Outcome	Status	Notable headers / body
Success	302 FOUND	Location: /dashboard, Set-Cookie: session=...
Failure	200 OK	login page HTML containing “invalid username or password”
Locked (defense on)	429 TOO MANY REQUESTS	login page HTML containing “account temporarily locked”

Table 2: HTTP response codes and bodies for each login outcome.

3.3 Design Notes, and What the Packet Capture Is Expected to Show

- Each attempt opens its own TCP connection, since the attacker scripts issue one HTTP request per attempt with no session reuse: a full three-way handshake, the HTTP request and response, then a FIN/ACK teardown, for every single attempt. Once the two-VM setup is ready, we will capture this in Wireshark and annotate it with the source and destination ports, sequence numbers, and the HTTP payload of each segment. *Follow > HTTP Stream* is the easiest way to show this in the report.
- Whether the username does not exist or the password is simply wrong, the response to a failed login is the same generic message, “invalid username or password”, so the app never leaks whether an account exists.
- Passwords are sent as cleartext form data over plain HTTP in this demo, with no TLS. This is a deliberate simplification, consistent with the assignment’s other plaintext-protocol attacks (Tool 2’s Telnet/HTTP sniffing, for example), and it is worth noting as a limitation compared to a production deployment, which would use HTTPS.

4 Implementation Plan

We plan to build the project in the following stages.

1. Victim webapp (Flask + SQLite).

- Signup endpoint: creates a new account, hashes and stores the password.
- Login endpoint: verifies submitted credentials and returns a distinct outcome for success, failure, and lockout (Section 3).
- Dashboard endpoint: reachable only after a successful login, so the “victim action” has something concrete to demonstrate.
- Two selectable hashing modes: a fast single SHA-256 hash (undefended baseline) and a salted PBKDF2 hash with a high iteration count (defended mode), toggled by one configuration flag so the same webapp can demonstrate both without code changes.

2. **Seed data.** A handful of demo accounts: some given passwords drawn from our dictionary wordlist, two sharing one default password (for the spray attack), and one given a strong, list-absent password as a negative control.
3. **Attacker tools.**
 - Dictionary-attack script: iterates a wordlist against one account via HTTP POST requests to `/login`, stopping on success or lockout.
 - Spray-attack script: iterates a small known-password list against many accounts, one password against all usernames before moving to the next.
 - Both scripts log every attempt (timestamp, username, password tried, response) for the Final Report.
4. **Defense mechanism.** A lockout tracker recording failed attempts per username and temporarily locking an account after a threshold is reached, plus the slow-hash mode described above (details in Section 7).
5. **Test environment.** We start development and functional testing on a single machine (localhost), to validate the protocol logic before introducing network variables, then move to the final demo on two VirtualBox VMs on a host-only network, matching the topology in Section 2, with a Wireshark capture on the link between them.
6. **Evaluation.** Run both attacks against the undefended webapp and record outcome/timing; enable the defense mode, reset the account store, and re-run the same attacks, recording the same metrics; compare the two runs for the Final Report.

5 Design Justification

Before presenting the evaluation in Section 6, we lay out here why we believe the design above should actually work.

- **The dictionary attack should succeed on a weak password.** Password-reuse and weak-password studies consistently find that a large share of real-world passwords are drawn from a comparatively small set of common strings, which is exactly what a dictionary attack exploits. The seeded accounts `alice`, `bob`, and `carol` use passwords (`sunshine`, `dragon`, `letmein`) that appear inside our 50-entry wordlist, so we expect the attack to find a match well before the list is exhausted. `dave`'s password, on the other hand, is high-entropy and absent from the wordlist by construction, so it should resist the attack. This negative control is what lets us attribute a successful crack to the password actually being weak, rather than to a bug in the attack script.
- **The spray attack should succeed independently of the dictionary.** `eve` and `frank` share the single password `admin123`, which does not need to appear anywhere in the 50-word dictionary at all. The spray attack only has to guess right once per *password*, amortized over every account it is tried against, so it can compromise both accounts in a single pass regardless of how the dictionary attack performs. This is why we see the two attacks as complementary rather than redundant: one exploits weak individual passwords, and the other exploits password reuse across accounts.
- **No raw packet or frame crafting should be required.** Unlike the link- and network-layer attacks elsewhere in the assignment (ARP poisoning, SYN flood, and so on), a login form is

an application-layer exploit. TCP already guarantees reliable, in-order delivery of the HTTP request, so the “exploit” lives entirely in the payload and server logic (Section 3), not in manipulating headers below layer 4. We consider this a natural consequence of the attack class, not a shortcut we are taking.

- **The lockout defense should stop the dictionary attack.** With the threshold set to $k = 5$ failed attempts per 30-second window, an attacker who does not already know the password gets, in expectation, only k guesses before being locked out for 60 seconds. Against a 50-word list, that caps the probability of success per lockout cycle at $k/50 = 10\%$ for a password placed uniformly at random in the list, compared to a near-certainty, given enough time, in the undefended case. That is why we expect the defended dictionary attack to be blocked rather than merely slowed down.
- **The slow hash should matter even when lockout does not.** PBKDF2-HMAC-SHA256 at 200,000 iterations costs on the order of 2×10^5 times as many underlying hash operations as the single SHA-256 hash used in the undefended mode. This is the standard argument for slow hashing: it raises the CPU cost of *every* guess by several orders of magnitude, which still matters in a scenario the per-account lockout cannot reach, namely an attacker who has exfiltrated the password-hash table and is guessing offline, with no server left to lock them out.
- **Failure responses should not leak account existence.** We return the same generic “invalid username or password” message (Section 3) whether the password is wrong or the username does not exist at all, and we hash against a dummy salt when the username is missing so that a nonexistent account does not respond measurably faster. Together these are meant to stop an attacker from using the login endpoint itself to enumerate valid usernames before attacking them.

6 Expected Outcome of a Successful Attack

Assuming the implementation works as intended, here is what we expect to see.

- **Dictionary attack.** For an account whose password is a common word included in our wordlist, we expect the attack to succeed within a modest number of attempts, a small fraction of the wordlist, returning a successful-login response and revealing the correct password. Attempts against the control account with a strong, list-absent password should exhaust the entire wordlist without success, confirming that the tool is working correctly and failing only because the password genuinely is not guessable this way, not because of a bug in the attack script.
- **Known-password (spray) attack.** For the two accounts seeded with the same default password, we expect a single spray pass to compromise both, without either password needing to appear in the larger dictionary wordlist. This is what demonstrates the attack exploiting password reuse specifically, as distinct from the dictionary attack’s brute-force search.
- **Undefended timing.** Without rate limiting, we expect the attacker to attempt logins about as fast as the network and a single fast hash allow, so a wordlist of realistic size (on the order of 50 to 100 entries) should be exhausted in well under a second, and the vulnerable accounts should be cracked almost immediately.
- **Defended timing.** With the lockout and slow-hash defense enabled, we expect each attempt to take noticeably longer, and we expect the dictionary attack to be cut short by the lockout, after a small, fixed number of failed attempts, well before it can exhaust even a modest wordlist.

In other words, a successful attack under the undefended configuration is expected to become a *blocked* attack under the defended one, given the same wordlist and target account.

7 Defense Idea

We build two complementary defenses, both togglable via the same “defense mode” flag on the webapp, so the exact same attacker scripts can be run against the undefended and defended configurations for comparison.

- 1. Account lockout.** The server tracks failed login attempts per username. After a small threshold of failures (e.g. 5) within a short time window (e.g. 30 seconds), the account is temporarily locked (e.g. for 60 seconds) and further attempts receive `429 TOO MANY REQUESTS` regardless of the password supplied. This directly caps how many guesses an attacker can make against a single account before being stopped, and blocks both the dictionary attack (many attempts, one account) and, per-account, the spray attack.
- 2. Slow password hashing.** In defended mode, passwords are hashed with PBKDF2-HMAC-SHA256 using a high iteration count (e.g. 200,000) instead of a single fast SHA-256 hash. This is a standard, well-documented technique for deliberately raising the CPU cost of each password check by one to two orders of magnitude, which slows down guessing even in scenarios (e.g. an offline attack on a stolen hash) where a lockout counter would not apply.

	Undefended	Defended
Cost per login attempt	fast (single SHA-256 hash)	significantly higher (PBKDF2, high iteration count)
Dictionary attack outcome	expected to succeed	expected to be blocked by lockout
Spray attack outcome	expected to succeed on reused-password accounts	expected to be throttled per account

Table 3: Expected before/after effect of the lockout + slow-hash countermeasure. Actual figures will be measured once the webapp and attacker scripts are implemented, and reported in the Final Report.

8 Conclusion

In this report, we set out to design a dictionary attack and a known-password (spray) attack against a signup and login web application we built ourselves, along with a defense meant to stop both. We believe the design holds together: the two attacks target genuinely different weaknesses, one preying on guessable individual passwords and the other on password reuse across accounts, the protocol they exploit needs nothing more than an ordinary HTTP POST request, and the lockout plus slow-hashing defense gives us a concrete, measurable way to show the same attack succeeding and then failing once the defense is switched on.

We have already built and tested a working prototype of the webapp, both attacker scripts, and the defense on a single machine, so we are reasonably confident the core logic works as intended. What remains before the Final Report is mostly a matter of scale and evidence: setting up the

two-VM network, capturing the traffic in Wireshark, and running the same attacks again so that the expected numbers in this report become measured ones.